SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 1 de 4 — Aceleradores de Hardware y Arquitectura de GPUs

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 1 de 4 — Aceleradores de Hardware y Arquitectura de GPUs

CPU frente a GPU en Cómputo de IA

De núcleos complejos optimizados para latencia secuencial a miles de núcleos paralelos SIMT

CPU: Diseñada para Baja Latencia

Pocos Núcleos Complejos
16 a 64 núcleos de alta frecuencia

Grandes Cachés SRAM (L2 / L3)
Predicción agresiva de ramas

Control de Flujo Complejo
Óptimo para tareas secuenciales y SO

GPU: Diseñada para Alto Throughput

Miles de Núcleos SIMT
Más de 10.000 ALUs aritméticas paralelas

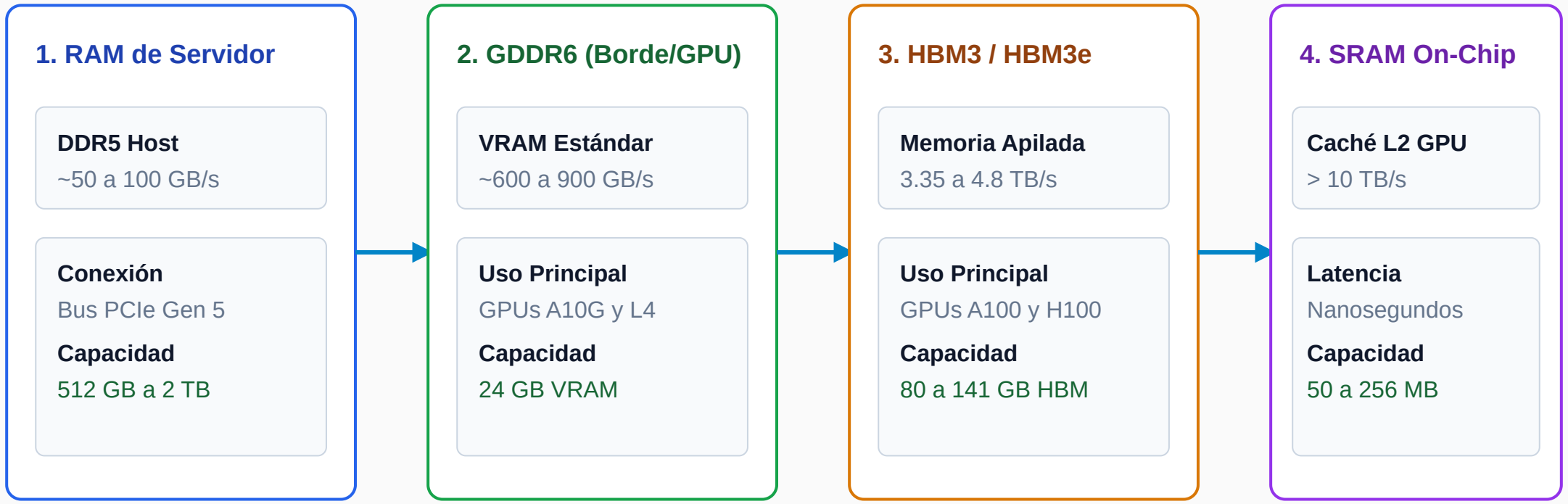
Mayor Espacio en Silicio para Cómputo
Cachés pequeñas, paralelismo masivo

Multiplicaciones Matriciales
Óptimo para transformaciones tensoriales

Fuente: NVIDIA Developer — GPU Architecture Overview (2025)

Jerarquía de Memoria en Aceleradores de IA

El verdadero cuello de botella en inferencia es transferir datos de memoria al silicio



Fuente: JEDEC — HBM3 Standard JESD238A & Micron Technology (2025)

Tensor Cores y Formatos Numéricos

De precisión simple FP32 a formatos especializados FP16, BF16 y FP8 para maximizar rendimiento

Formatos Numéricos en Inferencia

FP32 (32 bits = 4 bytes)
1 bit signo + 8 exponente + 23 mantisa (precisión base)

FP16 / BF16 (16 bits = 2 bytes)
Estándar de facto: 2x ahorro de memoria VRAM

FP8 (8 bits = 1 byte)
Nativa en Hopper/Blackwell: 4x ahorro de memoria

INT4 (4 bits = 0.5 bytes)
Compresión extrema para inferencia en un solo nodo

Mecanismo Tensor Core (MMA)

Operación Matricial: D = A × B + C

Multiplica dos matrices y acumula en una tercera

Ejecutada en un único ciclo de reloj por hardware

Aceleración Arquitectónica

Mayor densidad de cálculo por vatio consumido

Soporte directo en librerías cuBLAS y cuDNN

Fuente: IEEE Micro — NVIDIA Hopper Tensor Core Architecture (2024)

Interconexiones Multi-GPU: NVLink frente a PCIe

El paralelismo de tensores requiere comunicación inter-GPU sin pasar por el bus del procesador

PCIe Gen 5: Bus Compartido del Host

Ancho de Banda por Enlace x16
64 GB/s por dirección (128 GB/s bidireccional)

Cuello de Botella
El tráfico GPU-a-GPU satura el bus del CPU

Uso Recomendado
Modelos de un solo nodo GPU (ej. 8B en 1x A10G)

NVLink 4: Interconexión Punto a Punto

Ancho de Banda H100 / Blackwell
900 GB/s a 1.8 TB/s bidireccional por GPU

Conmutación NVSwitch
Acceso a memoria unificada entre 8 GPUs

Uso Obligatorio
Paralelismo tensorial en modelos de 70B a 405B

Fuente: NVIDIA — NVLink and NVSwitch Architecture Specification (2024)

Silicio Propietario de Nube: AWS Inferentia y Trainium

Alternativas a GPUs comerciales optimizadas para costo y rendimiento en cargas de trabajo de IA

1. AWS Inferentia 2 (Inf2)

Propósito de Diseño
Inferencia profunda de bajo costo

Chips NeuronCore-v2
32 GB de memoria HBM por chip

Beneficio Económico
Hasta 50 % menor costo

2. AWS Trainium (Trn1 / Trn2)

Propósito de Diseño
Entrenamiento y afinamiento masivo

Red NeuronLink-v2
Interconexión ultra-rápida entre chips

Eficiencia Energética
Optimizado para BF16 y FP8

3. SDK AWS Neuron

Compilación Ahead-of-Time
Optimiza grafos PyTorch y HF

Drivers y Runtime
Integrado con Kubernetes y ECS

Consideración de Nube
Menor costo; requiere compilación

Fuente: AWS Documentation — Annapurna Labs NeuronCore Overview (2025)

Familias de Instancias GPU en AWS EC2

Selección del tamaño de máquina adecuado según el tamaño del modelo y el volumen de tráfico

Familia G5 (NVIDIA A10G 24GB)

Perfil: Inferencia económica y modelos ligeros (hasta 8B)

Memoria: 24 GB GDDR6 por tarjeta | Bus: PCIe Gen 4

Ejemplo: g5.xlarge (~$1.00 USD/hora en US-East)

Casos: RAG corporativo, clasificación, modelos cuantizados

Familia G6 (NVIDIA L4 24GB)

Perfil: Eficiencia energética y bajo consumo (arquitectura Ada)

Memoria: 24 GB GDDR6 | Soporte: FP8 y decodificación de video

Ejemplo: g6.xlarge (~$0.80 USD/hora en US-East)

Casos: Visión por computador y modelos generativos medianos

Familia P4de (NVIDIA A100 80GB)

Perfil: Modelos fundacionales medianos y grandes (hasta 70B)

Memoria: 8x 80 GB HBM2e (640 GB total) | Red: 400 Gbps EFA

Ejemplo: p4de.24xlarge con interconexión NVLink

Casos: Inferencia de alta concurrencia y fine-tuning

Familia P5 (NVIDIA H100 80GB)

Perfil: Máximo rendimiento computacional y modelos de 405B

Memoria: 8x 80 GB HBM3 (640 GB) | Red: 3200 Gbps EFAv2

Ejemplo: p5.48xlarge con NVLink 4 (900 GB/s por GPU)

Casos: Razonamiento avanzado e inferencia distribuida masiva

Fuente: AWS Documentation — Amazon EC2 Accelerated Computing Instances (2025)

Redes de Ultra-Baja Latencia: EFA y RoCEv2

Transferencia directa de memoria entre servidores eliminando la sobrecarga del kernel del SO

Pila TCP/IP Clásica de Red

Múltiples Copias en Memoria
GPU VRAM a RAM de Host a Buffer de Kernel a NIC

Interrupciones del Sistema Operativo
Cambios de contexto y jitter en la red

Impacto en Inferencia
Latencias impredecibles que rompen SLAs interactivos

EFA y RDMA (Bypass de Kernel)

GPUDirect RDMA
Transferencia directa VRAM-a-VRAM sin CPU host

Protocolo SRD (Scalable Reliable Datagram)
Balanceo multicamino sin bloqueo de paquetes

Resultado Arquitectónico
Latencia sub-microsegundo y throughput consistente

Fuente: AWS Whitepaper — High Performance Computing on AWS & EFA Architecture (2024)

Particionamiento de GPU: NVIDIA Multi-Instance GPU (MIG)

División por hardware de una GPU de 80GB en hasta 7 instancias independientes con SLAs garantizados

NVIDIA A100 / H100 (80 GB VRAM Física) Dividida con MIG

MIG 1g.10gb

1 Slice Cómputo (14 SM)

10 GB VRAM Aislada

Uso: Modelo Embeddings

MIG 1g.10gb

1 Slice Cómputo (14 SM)

10 GB VRAM Aislada

Uso: Reranking Cross-Enc

MIG 2g.20gb

2 Slices Cómputo (28 SM)

20 GB VRAM Aislada

Uso: LLM Llama-3-8B

MIG 3g.40gb

3 Slices Cómputo (42 SM)

40 GB VRAM Aislada

Uso: Inferencia Batch 14B

Aislamiento Físico: Canales de memoria, decodificadores y cachés L2 completamente dedicados

Fuente: NVIDIA — Multi-Instance GPU User Guide (2025)

Eficiencia Energética y Refrigeración en Centros de Datos

Densidades de más de 40 kW por rack exigen transición de aire forzado a refrigeración líquida directa

Refrigeración por Aire Clásica

Límite Térmico Tradicional
10 a 15 kW por rack de servidores estándar

Desafío con Servidores de IA
Un solo servidor 8x H100 consume más de 10.2 kW

Consecuencia
Throttling térmico y degradación de frecuencias de reloj

Refrigeración Líquida Directa (DLC)

Direct-to-Chip (Cold Plates)
Líquido refrigerante fluye sobre la CPU y GPU

Capacidad por Gabinete
Soporta de 40 a más de 100 kW por rack

Eficiencia PUE
PUE cercano a 1.15 frente a 1.5 en refrigeración por aire

Fuente: Uptime Institute — Data Center Cooling & AI Power Density Trends (2025)

GPU Passthrough en Entornos OpenStack

Asignación directa de dispositivos PCI desde el hipervisor KVM a máquinas virtuales de cómputo

1. Host Físico (KVM / IOMMU)

Módulo VFIO-PCI
Desvincula la GPU del driver del host

Grupos IOMMU
Aislamiento seguro de memoria DMA

Configuración Nova
pci.passthrough_whitelist

2. Planificador de Nova

Filtro PCI en Scheduler
Ubica nodos con tarjetas libres

Flavor Especializado
extra_specs pci_passthrough:alias

Asignación Exclusiva
No admite sobreventa de hardware

3. Máquina Virtual Invitada

Visibilidad Nativa PCI
La VM ve el hardware de la tarjeta

Drivers NVIDIA Propios
Instala CUDA y vLLM sin parches

Rendimiento Bare-Metal
Pérdida menor al 1 % frente a físico

Fuente: OpenStack Docs — PCI Passthrough and SR-IOV Configuration (2025)

Rendimiento Real frente a Teórico: MFU y Throughput

Diferencia entre FLOPS pico en hoja técnica y la utilización efectiva real del silicio en inferencia

Métricas Teóricas vs Reales

TFLOPS Pico de Hoja Técnica
Capacidad máxima del silicio en condiciones ideales

MFU (Model FLOPs Utilization)
Ratio entre cálculos útiles ejecutados y capacidad pico

Rango Habitual en Inferencia
30 % a 55 % de aprovechamiento real del hardware

Causas de Pérdida de Eficiencia

Fase de Decodificación (Memory-Bound)
Lectura constante de pesos para generar un solo token

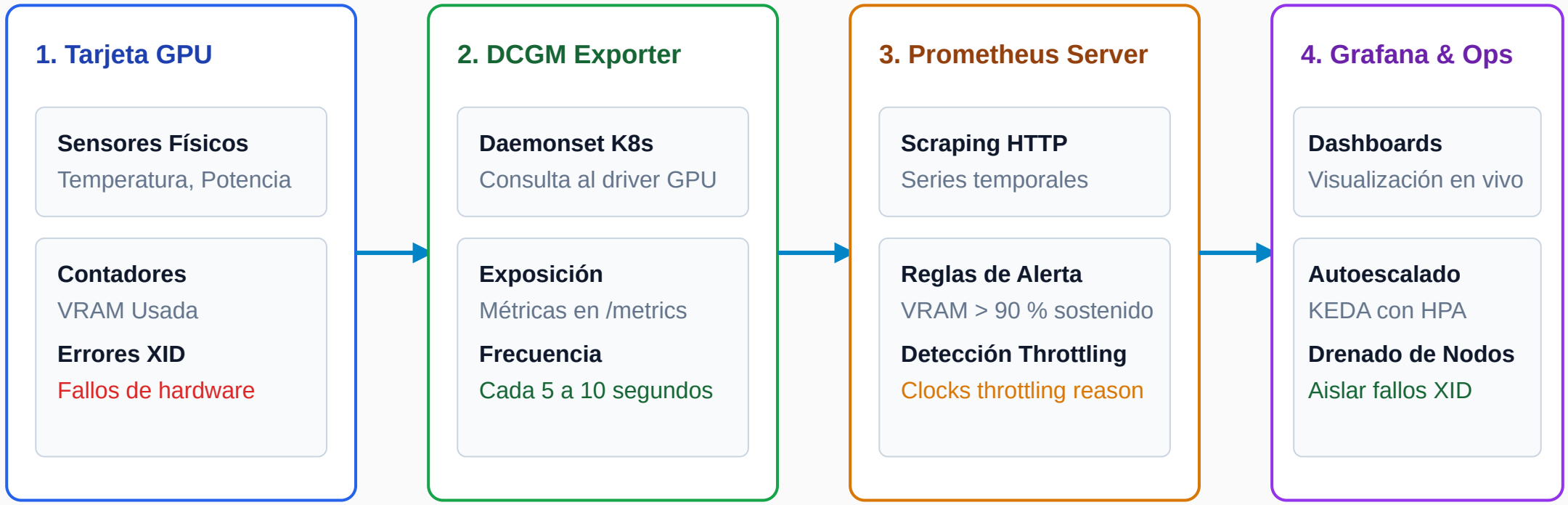
Burbujas en Lotes de Inferencia
Peticiones concurrentes con longitudes dispares

Latencia de Transferencia Host-to-Device
Copia de prompts por el bus PCIe

Fuente: Chowdhery et al. — PaLM: Scaling Language Modeling with Pathways (MFU Definition) (2024)

Monitorización de GPUs con NVIDIA DCGM

Métricas esenciales para detectar saturación de memoria, sobrecalentamiento y estrangulamiento térmico



Fuente: NVIDIA — Data Center GPU Manager (DCGM) User Guide (2025)

Caso Ilustrativo: Selección de Hardware para Inferencia

Caso ilustrativo de arquitectura, no es una empresa real — Matriz de decisión por tamaño de modelo

Escenario A: Modelo Ligero (8B)

Memoria Requerida en FP16
16 GB pesos + 4 GB KV-Cache = ~20 GB VRAM

Selección de Hardware
1x NVIDIA A10G (24 GB GDDR6) o 1x L4 (24 GB)

Resultado Arquitectónico
Instancia g5.xlarge, nodo único sin interconexión

Escenario B: Modelo Masivo (70B)

Memoria Requerida en FP16
140 GB pesos + 30 GB KV-Cache = ~170 GB VRAM

Selección de Hardware
4x NVIDIA A100 (80 GB) o 8x A10G con NVLink

Resultado Arquitectónico
Paralelismo tensorial obligatorio con enlace NVLink

Fuente: AWS Architecture Center — Generative AI Sizing Reference Pattern (2025)

SÍNTESIS CONCEPTUAL

Principios Clave de Cómputo Acelerado

Cuatro reglas fundamentales para el diseño de infraestructura física y en la nube para IA

1. Ancho de Banda

Memoria HBM3

La tasa de transferencia es el verdadero cuello de botella durante la decodificación

> 3 TB/s en A100/H100

2. Precisión Numérica

Formatos FP16 / FP8

Reducir los bits por peso duplica o multiplica la capacidad efectiva de la memoria VRAM

2x a 4x Throughput

3. Interconexión

NVLink y EFA

Bypass del bus PCIe y del kernel del SO para sincronización de tensores ultra-baja

Latencia sub-microseg

4. Particionamiento

NVIDIA MIG

División por hardware para evitar vecinos ruidosos y optimizar inversión en servidores

Hasta 7 VMs por GPU

Fuente: Síntesis de Arquitectura en la Nube — Universidad de la Costa (2026)

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 2 de 4 — Cuantización y Optimización de Modelos

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 2 de 4 — Cuantización y Optimización de Modelos

Presupuesto de Memoria VRAM en Inferencia

Desglose estricto de componentes para evitar errores de falta de memoria (Out-Of-Memory)

1. Pesos del Modelo

Consumo Fijo
Cargado al inicio

Fórmula Base
Bytes/param × Params

Ejemplo FP16
8B params = ~16 GB

2. Memoria KV-Cache

Consumo Dinámico
Crece por petición

Factores Clave
Longitud de contexto

Concurrencia
Usuarios simultáneos

3. Activaciones

Buffers Transitorios
Resultados de capas

Fase Crítica
Prefill del prompt

Impacto
Picos temporales

4. CUDA Overhead

Espacio del Driver
Contexto de GPU

Margen Seguro
1 a 2 GB reservados

Regla FinOps
No superar 90 % VRAM

Fuente: HuggingFace Documentation — Model Memory Estimation Guide (2025)

Fundamentos de Cuantización: De FP16 a Enteros

Mapeo de valores continuos de coma flotante a una cuadrícula discreta de menor resolución

Espacio Continuo FP16 (16 bits)

Resolución Numérica
65.536 valores distintos posibles por peso

Costo en Memoria
2 bytes por cada parámetro del modelo

Impacto en Servidores
Requiere clústeres multi-GPU para modelos de más de 30B

Espacio Cuantizado INT8 / INT4

Rejilla Discreta: INT8 (256 niveles) / INT4 (16)
Escala (Scale) y Punto Cero (Zero-Point)

Ahorro en VRAM
INT8 = 50 % reducción | INT4 = 75 % reducción

Consecuencia Arquitectónica
Un modelo de 70B pasa de 140 GB a solo 40 GB de VRAM

Fuente: Gholami et al. — A Survey of Quantization Methods for Efficient Neural Network Inference (2024)

Post-Training Quantization (PTQ) vs QAT

Compresión directa de modelos preentrenados frente a entrenamiento consciente de la cuantización

PTQ (Post-Training Quantization)

Metodología Operativa
Aplica sobre pesos ya entrenados sin reentrenar

Costo Computacional
Minutos a pocas horas en una sola GPU

Estándar en Producción
Métodos AWQ y GPTQ dominan la industria actual

QAT (Quantization-Aware Training)

Metodología Operativa
Simula errores de redondeo durante el entrenamiento

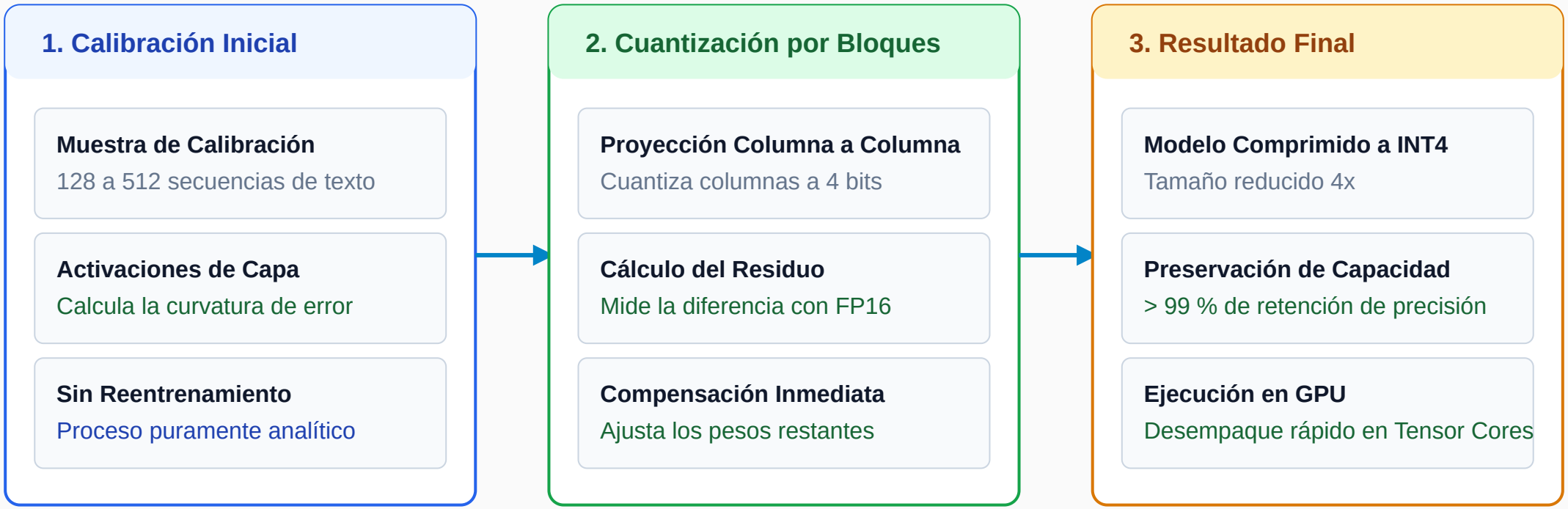
Costo Computacional
Requiere acceso a datos masivos y días de cómputo

Casos Específicos
Cuantización agresiva por debajo de 3 bits

Fuente: Jacob et al. — Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference (2024)

Algoritmo GPTQ: Compensación de Error por Capas

Ajuste dinámico de los parámetros no cuantizados para neutralizar la pérdida de precisión



Fuente: Frantar et al. — GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (2023)

Algoritmo AWQ: Protección de Canales Salientes

El 1 % de los parámetros concentra la inteligencia del modelo y debe protegerse de la compresión

Cuantización Ingenua Uniforme

Tratamiento Igualitario
Todos los pesos se redondean bajo la misma escala

Problema de los Outliers
Un 1 % de pesos tiene activaciones 100x mayores

Consecuencia Crítica
Degradación catastrófica de razonamiento y lógica

Mecanismo AWQ (Activation-Aware)

Identificación de Canales Salientes
Monitorea la magnitud de las activaciones reales

Escalado Selectivo por Hardware
Multiplica pesos críticos por un factor de protección

Ventaja en Inferencia
Mayor velocidad que GPTQ en servidores vLLM

Fuente: Lin et al. — AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration (2024)

Formato GGUF y Ecosistema llama.cpp

Inferencia híbrida dividiendo capas del modelo entre CPU de servidor y memoria de tarjeta gráfica

Estructura del Archivo Binario GGUF

Encapsulado Único Autocontenido
Contiene metadatos, vocabulario del tokenizador y pesos

Mapeo Directo en Memoria (mmap)
Carga instantánea de disco a RAM sin deserialización

Variantes K-Quants
Niveles Q4_K_M y Q5_K_M adaptan bits según la capa

Ejecución Híbrida: GPU Offloading

División Flexible de Capas
20 capas en VRAM (GPU) + 12 capas en RAM (CPU)

Sin Dependencias Pesadas
Escrito en C/C++ nativo sin necesidad de Python

Uso Arquitectónico
Dispositivos en el borde (Edge), PCs de desarrollo y Ollama

Fuente: Gerganov et al. — llama.cpp & GGUF Binary Specification (2025)

Métricas de Degradación: Perplejidad y Benchmarks

Evaluación rigurosa de la pérdida de calidad sintáctica y de razonamiento tras la compresión

Perplejidad (PPL): Evaluación Sintáctica

Concepto Matemático
Incertidumbre al predecir el próximo token

Regla de Interpretación
Menor perplejidad equivale a mayor fluidez y precisión

Umbral de Aceptación en Nube
Incremento menor a 0.2 puntos frente a FP16 original

Benchmarks de Razonamiento (MMLU / GSM8K)

Evaluación Multitarea (MMLU)
Pruebas en derecho, medicina, ciencias e ingeniería

Razonamiento Matemático (GSM8K)
Sensible a errores en representaciones numéricas

Criterio Arquitectónico
Aceptar cuantización solo si retiene > 98 % del benchmark

Fuente: Hendrycks et al. — Measuring Massive Multitask Language Understanding (MMLU) (2024)

Poda (Pruning) y Dispersión Estructurada 2:4

Aceleración por hardware eliminando pesos redundantes en bloques regulares de matrices

Dispersión No Estructurada (Unstructured)

Eliminación Arbitraria de Pesos

Convierte en cero los parámetros con menor magnitud

Desafío en Aceleradores

Crea patrones irregulares que rompen la ejecución SIMT

Resultado Práctico

Ahorra espacio en disco pero no acelera la inferencia

Dispersión Estructurada 2:4 (NVIDIA Sparse)

Patrón Regular por Hardware

Dos de cada cuatro valores contiguos son cero

Aceleración Nativa en Tensor Cores

La GPU omite los ceros y duplica el rendimiento de cálculo

Beneficio Arquitectónico

Hasta 2x mayor throughput con mínima degradación

Fuente: NVIDIA Developer — Structured Sparsity in Ampere and Hopper Architecture (2024)

Destilación de Conocimiento: Profesor a Estudiante

Transferencia de la distribución probabilística de un modelo masivo a un modelo compacto

1. Modelo Profesor (Teacher)

Gran Escala (70B a 405B)
Razonamiento y alta capacidad

Genera 'Soft Labels'
Probabilidades continuas

Temperatura Elevada
Revela relaciones semánticas sutiles

2. Proceso de Transferencia

Alineación de Probabilidades
Minimiza divergencia KL de salida

Entrenamiento Eficiente
Aprende qué es probable y qué no

Datos Sintéticos
Generados por el propio profesor

3. Modelo Estudiante (Student)

Escala Compacta (1B a 8B)
Opera en una sola GPU económica

Velocidad de Inferencia
5x a 10x mayor generación de tokens

Eficiencia en la Nube
Costo operativo mucho menor

Fuente: Hinton et al. — Distilling the Knowledge in a Neural Network & TinyLlama Project (2024)

Evolución de la Atención: MHA, MQA y GQA

Reducción drástica del tamaño de la memoria KV-Cache compartiendo cabezas de claves y valores

Multi-Head (MHA)

Estructura 1:1:1
Cada Query tiene su Key y Value

Consumo de KV-Cache
8x a 32x mayor consumo de VRAM

Modelos Históricos
GPT-3 y Llama-1 original

Multi-Query (MQA)

Estructura N:1:1
Querys comparten 1 Key y Value

Máxima Compresión
Reduce la caché KV hasta un 90 %

Compromiso
Pérdida perceptible en razonamiento

Grouped-Query (GQA)

Estructura Agrupada
8 grupos de Querys comparten 1 KV

Equilibrio Perfecto
8x ahorro en KV sin perder precisión

Estándar Industrial
Usado en Llama-3, Mistral y Claude

Fuente: Ainslie et al. — GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints (2023)

Cuantización de la KV-Cache: FP8 e INT8

Compresión en tiempo real de los estados pasados de la conversación en contextos de más de 32k tokens

KV-Cache Estándar FP16 (2 Bytes/Token)

Presupuesto por Petición

32k tokens de contexto consumen ~2 GB por usuario

Límite de Concurrencia

GPU de 24 GB satura con 4 consultas

Consecuencia

Rechazo de peticiones por falta de memoria (OOM)

KV-Cache Cuantizada FP8 / INT8

Ahorro Inmediato del 50 %

Cada token de contexto pasa de 2 bytes a solo 1 byte

Duplicación de Throughput

Permite atender el doble de usuarios en el mismo hardware

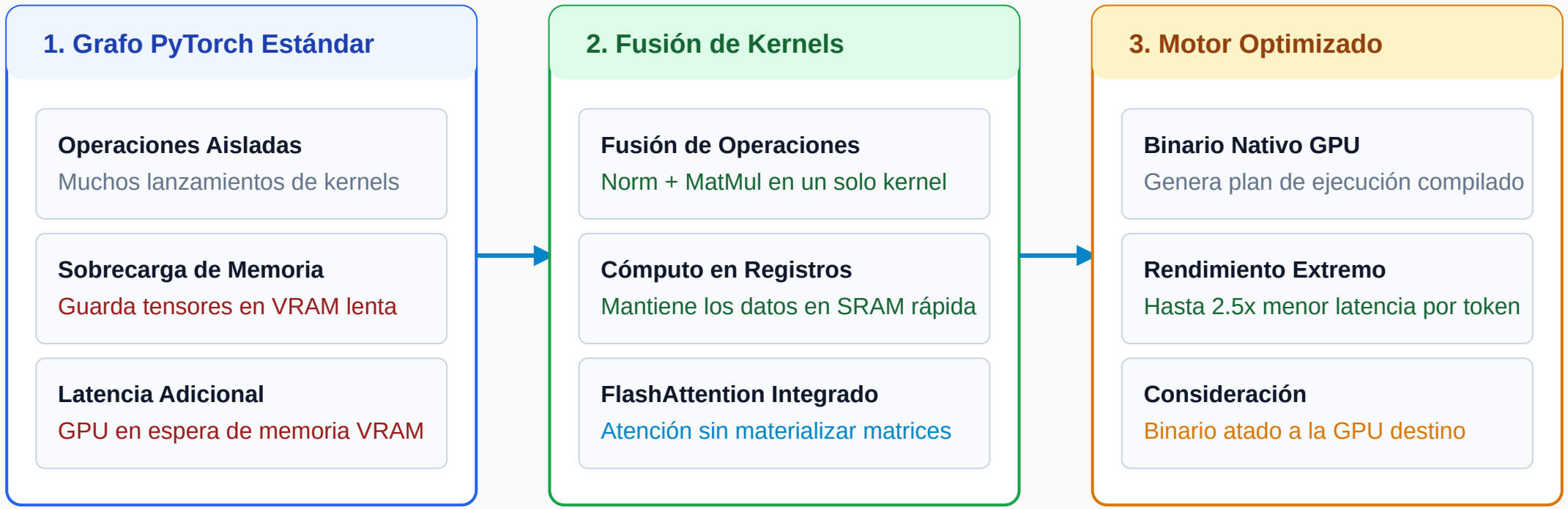
Soporte Nativo en Motores

Activación transparente vía parámetro --kv-cache-dtype fp8

Fuente: vLLM Documentation — KV-Cache Quantization (FP8 and INT8) Benchmarks (2025)

Compiladores de Inferencia: NVIDIA TensorRT-LLM

Fusión de operaciones matemáticas y optimización de kernels a nivel de ensamblador de GPU



Fuente: NVIDIA Developer — TensorRT-LLM Architecture and Kernel Fusion (2025)

Caso Ilustrativo: Compresión de Llama-70B

Caso ilustrativo de arquitectura, no es una empresa real — Reducción de 140 GB a 40 GB VRAM

Configuración Original FP16

Tamaño de Pesos

70 mil millones × 2 bytes = 140 GB VRAM

Infraestructura Requerida

4x tarjetas A100 (80 GB) con NVLink obligatorio

Costo Operativo

Instancia p4de.24xlarge (~$15 a $20 USD/hora)

Configuración Cuantizada AWQ INT4

Tamaño de Pesos

70 mil millones × 0.5 bytes = ~35 a 40 GB VRAM

Infraestructura Requerida

2x NVIDIA A10G (24 GB) o 1x A100 (80 GB)

Ahorro Económico

Instancia g5.12xlarge (~$5.60 USD/hora, > 65 % ahorro)

SÍNTESIS DE MÉTODOS

Comparativa de Métodos de Cuantización

Matriz de selección arquitectónica según hardware disponible, formato de archivo y motor de inferencia

AWQ (INT4)

Enfoque Clave
Protege canales salientes críticos

Motor Óptimo
vLLM en GPUs cloud

Recomendación
Servidores de nube

GPTQ (INT4)

Enfoque Clave
Compensación analítica de error por capas

Motor Óptimo
AutoGPTQ / vLLM

Recomendación
Inferencia por lotes

GGUF (K-Quants)

Enfoque Clave
Cuantización mixta archivo autocontenido

Motor Óptimo
llama.cpp / Ollama

Recomendación
Edge y entorno local

FP8 Nativo

Enfoque Clave
Formato flotante de 8 bits en silicio

Motor Óptimo
TensorRT-LLM

Recomendación
Tarjetas Ada / Hopper

Fuente: Guía de Selección de Cuantización para Arquitectura de IA — CUC (2026)

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

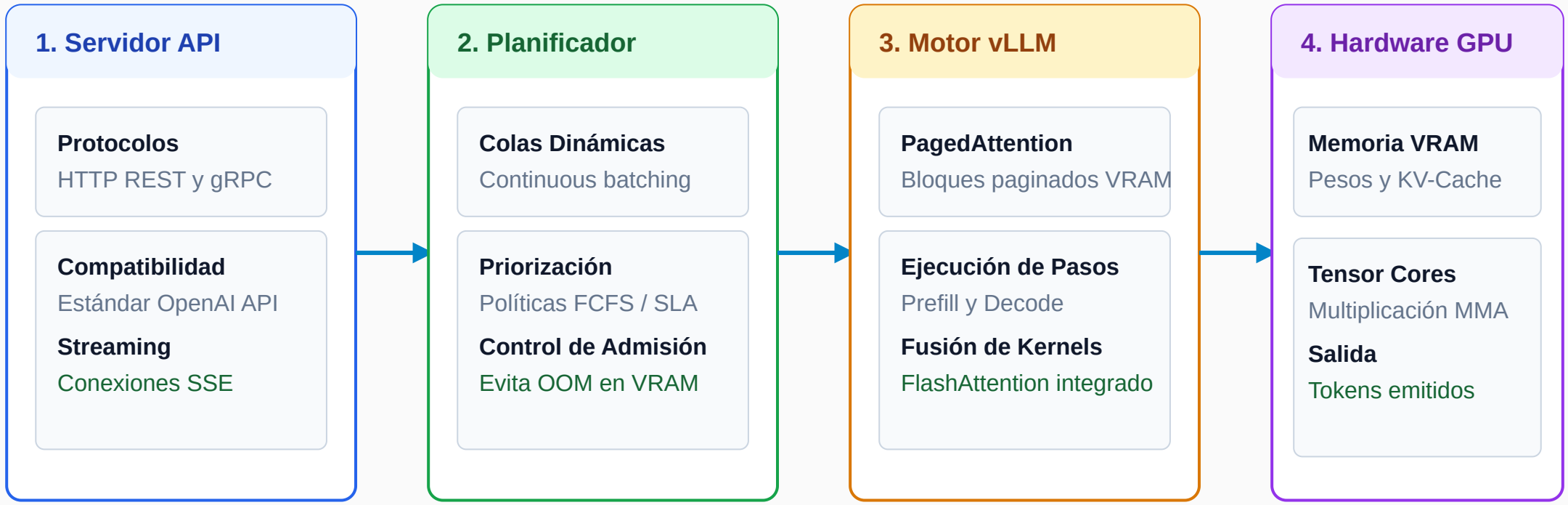
Parte 3 de 4 — Servidores de Inferencia Auto-Alojados (Self-Hosted)

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 3 de 4 — Servidores de Inferencia Auto-Alojados (Self-Hosted)

Anatomía de un Servidor de Inferencia Moderno

Desacoplamiento entre la capa de red, el planificador de lotes y el motor de ejecución en GPU



Fuente: Kwon et al. — Efficient Memory Management for Large Language Model Serving with PagedAttention (2023)

vLLM y PagedAttention: Cero Fragmentación

Inspirado en la memoria virtual paginada de los sistemas operativos para maximizar la VRAM

Asignación Contigua Tradicional

Preasignación Estática
Reserva memoria para la longitud máxima (ej. 4k tokens)

Fragmentación Interna Grave
Si la respuesta es corta, el 80 % del espacio queda ocioso

Desperdicio de VRAM
Hasta 60 % a 80 % de memoria inutilizada en producción

Mecanismo PagedAttention (vLLM)

Bloques Lógicos de Tokens
Divide la caché en bloques fijos de 16 tokens

Tabla de Páginas Dinámica
Mapea bloques no contiguos en cualquier región de VRAM

Resultado Arquitectónico
Cero fragmentación y hasta 4x mayor throughput

Fuente: Kwon et al. — PagedAttention Paper, SOSP 2023 & UC Berkeley Sky Computing Lab (2024)

Continuous Batching (In-Flight Batching)

Inserción y retiro de peticiones a nivel de token individual en cada iteración del ciclo

Lotes Estáticos Clásicos

Sincronización Rígida
El lote espera a que todas las peticiones terminen

Subutilización Masiva
Peticiones cortas esperan a las largas

Impacto en Latencia
Aumenta drásticamente la latencia media y P99

Continuous Batching (Orca / vLLM)

Planificación por Iteración
El lote se reconfigura en cada emisión de token

Entrada y Salida Dinámica
Peticiones completadas salen; nuevas peticiones entran

Aprovechamiento de Silicio
Mantiene los Tensor Cores ocupados > 80 % del tiempo

Fuente: Yu et al. — Orca: A Distributed Serving System for Transformer-Based Generative Models (OSDI 2022 / vLLM 2024)

Decodificación Especulativa (Speculative Decoding)

Modelo borrador rápido propone candidatos y el modelo principal verifica en una pasada

1. Modelo Borrador (Draft)

Modelo Pequeño (ej. 1B)
Ejecución ultra-rápida y ligera

Emisión de K Candidatos
Propone una ráfaga de 3 a 5 tokens

Bajo Costo VRAM
Comparte GPU con modelo mayor

2. Verificación en Paralelo

Modelo Principal (70B)
Evalúa los K tokens simultáneamente

Cómputo en Prefill
Multiplica matrices en una pasada

Regla de Aceptación
Acepta tokens correctos en bloque

3. Rendimiento Resultante

Aceleración 2x a 3x
Emite múltiples tokens por ciclo

Cero Pérdida de Calidad
Distribución idéntica al modelo 70B

Ahorro en Hardware
Menor latencia sin cambiar tarjetas

Fuente: Leviathan et al. — Fast Inference from Transformers via Speculative Decoding (Google Research 2023)

SELECCIÓN DE MOTOR

Comparativa de Motores: vLLM, TGI y Triton

Evaluación de las plataformas de inferencia dominantes en despliegues auto-alojados

vLLM (UC Berkeley)

Punto Fuerte
Mayor throughput concurrente

Ecosistema
Python nativo y comunidad activa

Recomendación
Opción líder para LLMs en nube

HuggingFace TGI

Punto Fuerte
Control estricto de parada y tokens

Arquitectura
Servidor Rust con gRPC ultra-rápido

Recomendación
Integración nativa en SageMaker

NVIDIA Triton

Punto Fuerte
Máxima optimización para NVIDIA

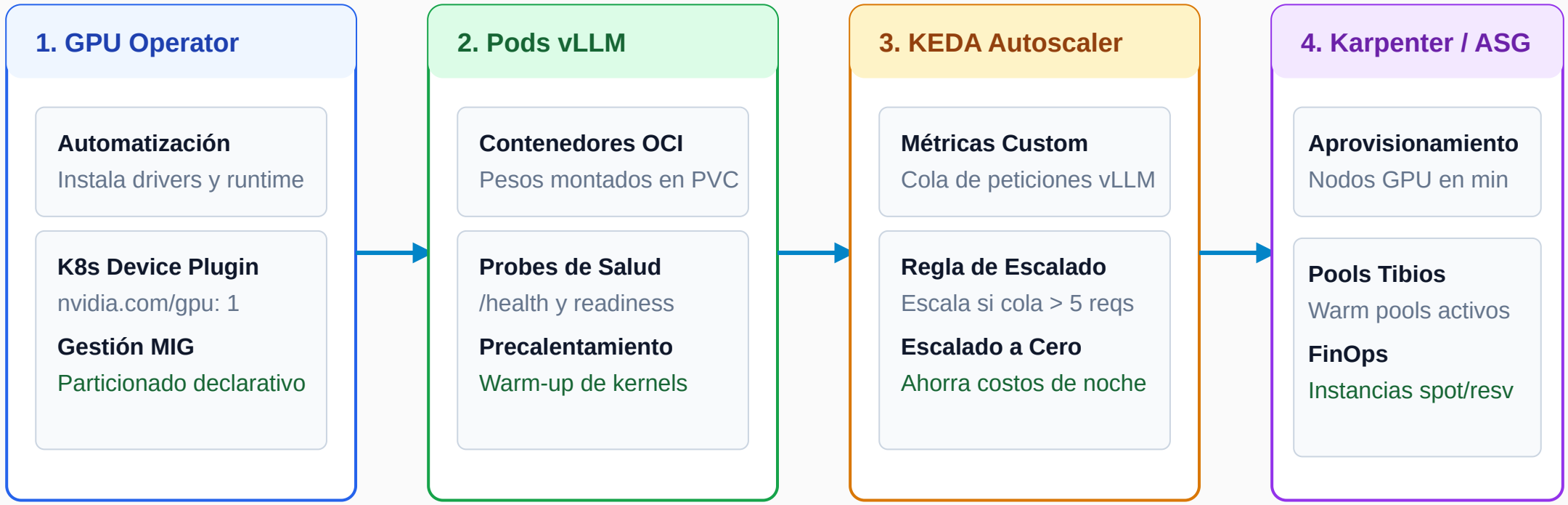
Capacidades
Soporta TensorRT-LLM y ONNX

Recomendación
Pipelines multimodales complejos

Fuente: Benchmarks Oficiales de Rendimiento de Inferencia — MLCommons / Sky Computing (2025)

Despliegue en Kubernetes con GPU Operator y KEDA

Autoescalado horizontal basado en la profundidad de colas y utilización real de VRAM



Fuente: CNCF — Kubernetes GPU Scheduling & KEDA AI Scaler Patterns (2025)

Paralelismo Tensorial y de Pipeline

Técnicas de partición de modelos masivos a través de múltiples GPUs y múltiples servidores

Paralelismo Tensorial (Tensor Parallel)

Partición Intra-Capa
Divide las matrices de atención y FFN en cada GPU

Requisito de Red Estricto
Exige NVLink (> 900 GB/s) por All-Reduce

Ámbito de Despliegue
Dentro de un solo servidor físico (hasta 8 GPUs)

Paralelismo de Pipeline (Pipeline Parallel)

Partición Inter-Capa
Capas 1-20 en Servidor A; capas 21-40 en Servidor B

Requisito de Red Moderado
Soporta redes de datos 100/400 Gbps con EFA

Compromiso Arquitectónico
Introduce burbujas de latencia si no se satura el pipeline

Fuente: Shoeybi et al. — Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism (2024)

Métricas de Servicio: TTFT, TPOT y Throughput

Los tres indicadores determinantes para definir acuerdos de nivel de servicio en inferencia

1. TTFT (Time to First Token)

Definición
Desde envío hasta primer byte

Fase Determinante
Fase de prefill (procesar prompt)

Objetivo SLA
< 800 ms para chat interactivo

2. TPOT (Time Per Output Token)

Definición
Tiempo entre tokens generados

Fase Determinante
Fase de decodificación iterativa

Objetivo SLA
< 40 ms/token (> 25 tokens/seg)

3. Throughput Global

Definición
Total de tokens servidos por segundo

Métrica FinOps
Determina el costo por token real

Objetivo de Clúster
> 1.500 tokens/seg por nodo GPU

Fuente: AWS Generative AI Lens — SLA and Performance Tuning for LLMs (2025)

ESTABILIDAD OPERATIVA

Control de Latencia P99: Chunked Prefill

Partición de prompts gigantescos en bloques para evitar congelamientos en peticiones concurrentes

Sin Chunked Prefill (Monolítico)

Llegada de Prompt Largo (16k tokens)
El motor procesa el prompt entero de golpe

Monopolización de la GPU
La tarjeta dedica el 100 % de núcleos durante 1.5 seg

Efecto en Usuarios Activos
El streaming de chat interactivo se congela (Jitter P99)

Con Chunked Prefill (Sarathi / vLLM)

División en Trozos (ej. 512 tokens)
El prompt se divide en fragmentos regulares

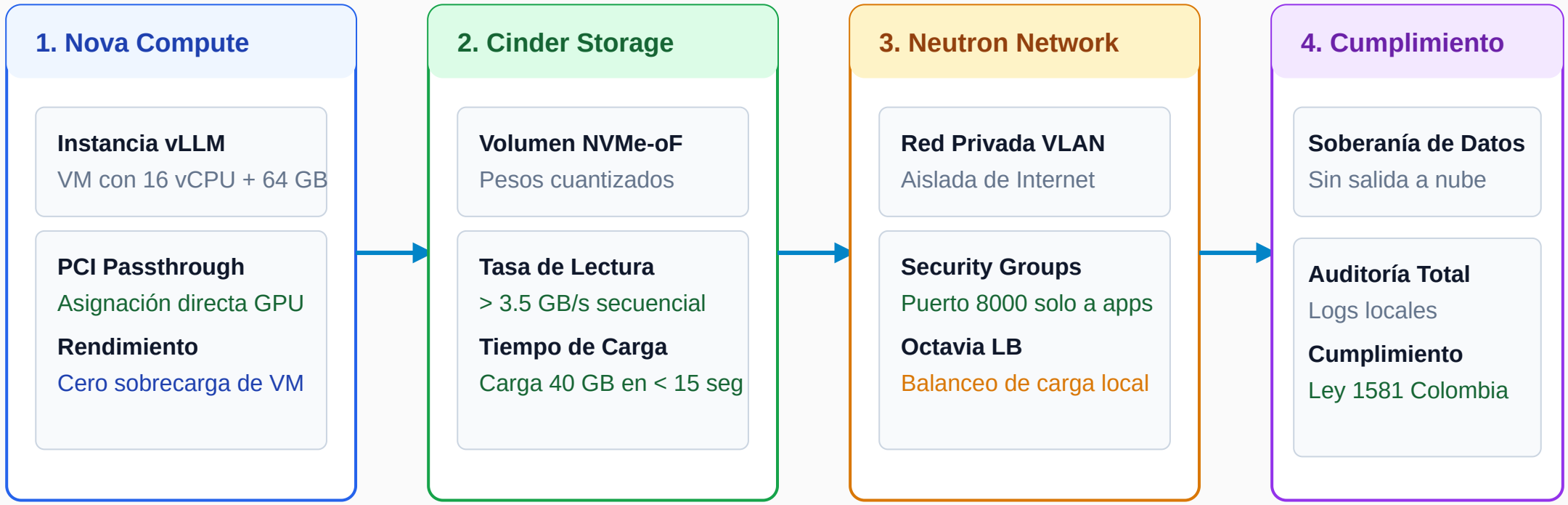
Coexistencia en el Lote
Cada paso ejecuta 1 trozo de prefill + decodificaciones

Estabilidad de Latencia P99
Garantiza emisión continua de texto sin pausas

Fuente: Agrawal et al. — Sarathi: Efficient LLM Inference by Chunked-Prefills (2024)

vLLM en Nube Privada OpenStack

Aprovisionamiento de servidores de inferencia sobre hipervisores KVM con almacenamiento Cinder



Fuente: Guía de Arquitectura de IA en OpenStack — Laboratorio CUC (2026)

Recuperación de Fallos y Errores XID de GPU

Detección temprana de anomalías en silicio y aislamiento automático de nodos degradados

Errores Críticos XID de Hardware

XID 31 (Fallo de Memoria GPU)
Fallo irrecuperable de página en bus HBM/GDDR

XID 45 (Caída de Enlace PCIe / NVLink)
Desconexión física o error fatal en bus de comunicación

XID 79 (Fallo Térmico / Alimentación)
Apagado de seguridad por temperatura > 95 °C

Mecanismo de Recuperación Automática

Detección Inmediata por DaemonSet
Lee eventos del kernel (/var/log/messages)

Drenado Seguro de Nodo (Cordon & Drain)
El balanceador retira el tráfico del nodo afectado

Reemplazo Transparente
El Auto Scaling Group lanza un nuevo servidor sano

Fuente: NVIDIA Developer — GPU XID Error Code Reference Guide (2025)

Auditoría de Tokens y FinOps en IA

Control de costos unitarios y atribución de consumo por centro de costos en servidores propios

1. Atribución de Consumo

Etiquetado por API Key
Identifica usuario y departamento

Métricas en Gateway
Mide prompt y completion tokens

Showback / Chargeback
Imputación presupuestal mensual

2. Cálculo de Costo Unitario

Fórmula de Costo Real
Costo Servidor / Millones Tokens

Tasa de Ocupación
A mayor concurrencia, menor costo

Comparativa con APIs
Valida la rentabilidad del hardware

3. Prevención de Abuso

Cuotas por Aplicación
Techos máximos de tokens al mes

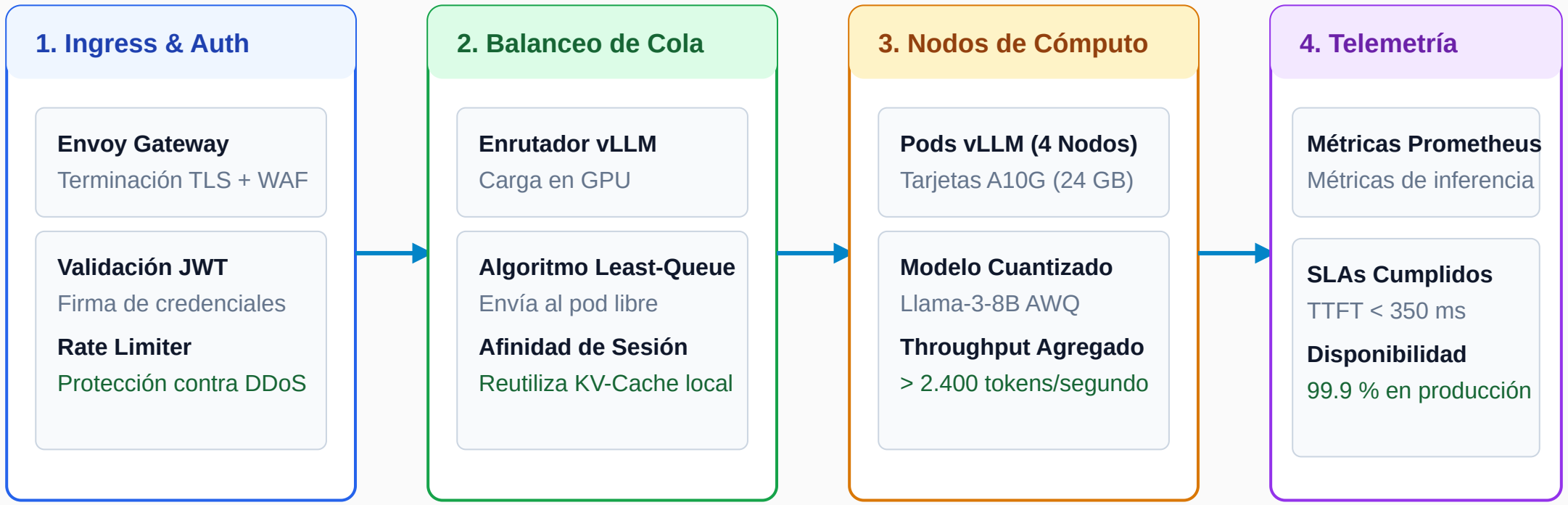
Detección de Bucles
Frena agentes en bucles infinitos

Alarmas en Tiempo Real
Notificación ante picos anómalos

Fuente: FinOps Foundation — AI and Machine Learning FinOps Framework (2025)

Caso Ilustrativo: Arquitectura Self-Hosted de Producción

Caso ilustrativo de arquitectura, no es una empresa real — Flujo integral de inferencia en clúster privado



Fuente: Patrón de Referencia de Servidores de Inferencia en Producción — CUC (2026)

CHECKLIST DE OPERACIONES

Decálogo de Operación para Inferencia Self-Hosted

Buenas prácticas indispensables para mantener servidores de inferencia estables y económicos

1. Memoria VRAM

Regla del 90 %

Fijar límite seguro de memoria para evitar Out-Of-Memory en el runtime CUDA

gpu_memory_util: 0.90

2. Planificación

Continuous Batch

Habilitar siempre colas dinámicas y Chunked Prefill para estabilizar P99

PagedAttention On

3. Telemetría

DCGM + KEDA

Monitorear errores XID, temperatura y profundidad de cola para autoescalar

Alertas Proactivas

4. Resiliencia

Drenado Automático

Aislar tarjetas con fallos de hardware y contar con pools tibios de respaldo

Alta Disponibilidad

Fuente: Síntesis de Operaciones de IA — Universidad de la Costa (2026)

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 4 de 4 — Servicios Gestionados vs Auto-Alojados y TCO

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 4 de 4 — Servicios Gestionados vs Auto-Alojados y TCO

DILEMA ARQUITECTÓNICO

El Dilema: Gestionado frente a Auto-Alojado

Comparativa de responsabilidades, costos fijos vs variables y tiempo de salida al mercado

Servicios Gestionados (MaaS - Bedrock)

Modelo de Costos
100 % variable por token (cero costo base)

Carga Operativa
Mínima: El proveedor gestiona parches y GPUs

Tiempo de Salida (Time-to-Market)
Inmediato: Acceso instantáneo a modelos de frontera

Servidores Dedicados (Self-Hosted)

Modelo de Costos
Costo fijo mensual por servidor reservado

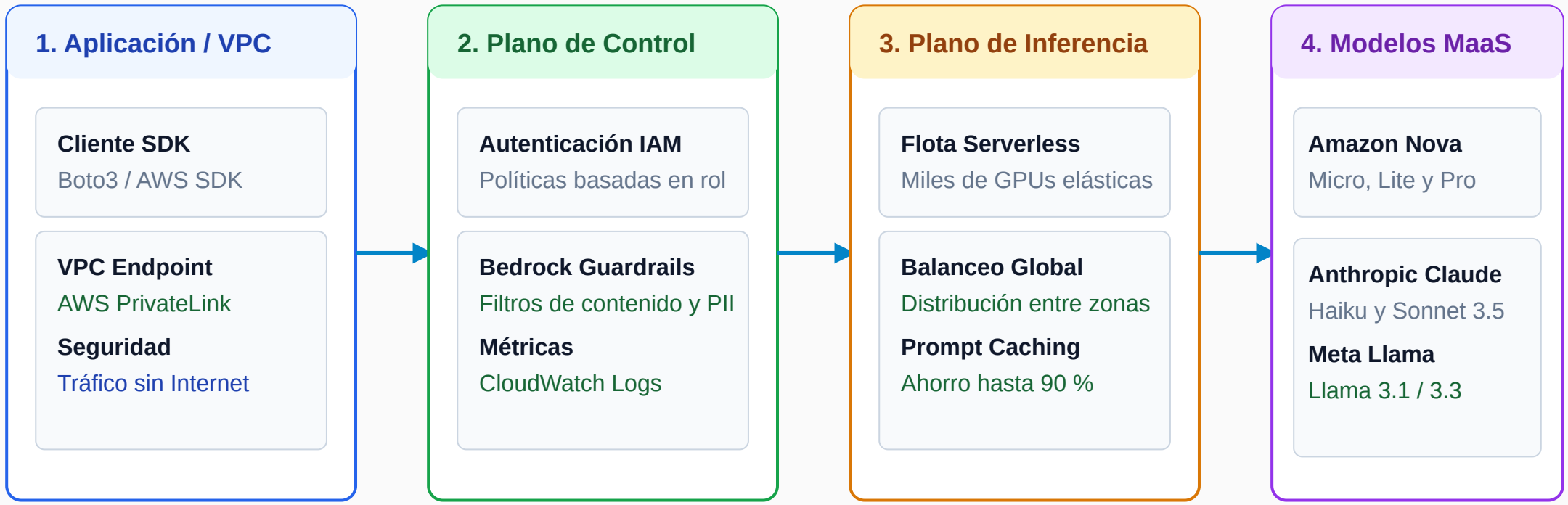
Carga Operativa
Alta: Mantenimiento de CUDA, vLLM y fallos XID

Control y Privacidad
Total: Modelos propios y datos sin salir de la red

Fuente: AWS Prescriptive Guidance — Choosing Between Managed and Self-Hosted LLMs (2025)

Arquitectura de Amazon Bedrock

Punto de acceso serverless unificado con aislamiento multi-tenant y sin gestión de servidores



Fuente: AWS Documentation — Amazon Bedrock User Guide & Security Architecture (2025)

MODALIDADES DE CONTRATACIÓN

On-Demand frente a Capacidad Aprovisionada

Pago elástico por token individual frente a reserva de unidades de rendimiento (PTUs)

Modalidad On-Demand (Bajo Demanda)

Mecanismo de Cobro

Facturación por millón de tokens procesados

Elasticidad y Compromiso

Cero compromiso temporal; escala a cero en inactividad

Límites Operativos

Sujeto a cuotas de tokens por minuto (TPM) compartidas

Capacidad Aprovisionada (Provisioned Throughput)

Mecanismo de Cobro

Tarifa horaria por Unidad de Rendimiento (Model Unit)

Garantía de Rendimiento

Throughput reservado y latencia ultra-consistente

Compromiso Contractual

Compromisos de 1 mes o 6 meses para modelos propios

Fuente: AWS Bedrock Documentation — Provisioned Throughput Guidelines (2025)

Aislamiento y Retención Cero de Datos (ZDR)

Garantías contractuales y técnicas de que los datos no se utilizan para entrenar modelos

1. Zero Retention

Sin Almacenamiento
Los prompts y respuestas no se guardan en disco por parte del proveedor

Sin Reentrenamiento
Nunca se usan datos para entrenar modelos

2. Cifrado Total

En Tránsito (TLS 1.3)
Túneles cifrados de extremo a extremo

En Reposo (KMS)
Llaves propias CMK gestionadas en AWS KMS

Rotación de llaves

3. Aislamiento VPC

AWS PrivateLink
Endpoints dentro de la subred corporativa

Sin Salida Pública
No requiere Internet Gateway ni NAT

Control por IAM

4. Certificaciones

Cumplimiento Global
SOC 1, SOC 2, SOC 3
Certificación ISO 27001

Sector Salud
Elegible para HIPAA

Sector Financiero
PCI-DSS Nivel 1

Fuente: AWS Bedrock Security & Data Protection Policy (2025)

Gestión de Cuotas y Límites de Tasa en MaaS

Políticas de protección contra costos imprevistos y estrangulamiento de peticiones (Throttling)

Límites Operativos de la API

RPM (Requests Per Minute)
Tope de peticiones HTTP concurrentes permitidas

TPM (Tokens Per Minute)
Tope de volumen agregado de tokens por minuto

Respuesta ante Sobrecarga
Error HTTP 429 (ThrottlingException)

Patrones de Resiliencia en el Cliente

Reintentos con Retroceso Exponencial
Exponential Backoff con Jitter aleatorio

Colas de Desacoplamiento (SQS)
Amortigua picos de tráfico sin saturar la API

Presupuestos AWS Budgets
Alertas al alcanzar 80 % del presupuesto mensual

Fuente: AWS Bedrock Quotas and Service Limits Documentation (2025)

Responsabilidad Compartida en Inteligencia Artificial

Delimitación clara entre el proveedor de nube, el creador del modelo y el arquitecto consumidor

1. Proveedor de Nube (AWS)

Seguridad Física
Centros de datos y hardware GPU

Infraestructura Base
Aislamiento de red y virtualización

Disponibilidad
SLA de API (99.9 % uptime)

2. Proveedor del Modelo

Preentrenamiento
Curaduría ética de datos base

Alineación (RLHF)
Mitigación de sesgos y toxicidad

Pesos del Modelo
Integridad de arquitectura base

3. Cliente / Arquitecto

Datos y Prompts
Clasificación de confidencialidad

Protección Prompt Injection
Filtros de entrada y validación

Uso Final y Salidas
Verificación de alucinaciones

Fuente: AWS Shared Responsibility Model for Generative AI (2025)

Estructura de Precios Reales en Amazon Bedrock

Tarifas por millón de tokens en región US-East (N. Virginia), vigentes a noviembre de 2026

Amazon Nova Micro (Modelo Ultraligero)

Entrada (Input): $0.035 USD por 1M tokens

Salida (Output): $0.140 USD por 1M tokens

Descuento Batch: 50 % ($0.0175 / $0.0700)

Caso de uso: Clasificación, extracción y enrutamiento rápido

Amazon Nova Lite (Multimodal Eficiente)

Entrada (Input): $0.060 USD por 1M tokens

Salida (Output): $0.240 USD por 1M tokens

Prompt Caching: $0.015 USD por 1M lectura de caché

Caso de uso: Chat interactivo multimodal y agentes ligeros

Claude 3.5 Haiku (Alta Velocidad)

Entrada (Input): $0.800 USD por 1M tokens

Salida (Output): $4.000 USD por 1M tokens

Prompt Caching: $0.080 USD por 1M lectura (90 % ahorro)

Claude 3 Haiku ($0.25/$1.25) rotulado modelo histórico

Claude 3.5 Sonnet (Razonamiento de Frontera)

Entrada (Input): $3.000 USD por 1M tokens

Salida (Output): $15.000 USD por 1M tokens

Prompt Caching: $0.300 USD por 1M lectura (90 % ahorro)

Uso: Programación, análisis y agentes avanzados

Fuente: AWS Bedrock Official Pricing (Consultado Noviembre 2026)

Punto de Equilibrio: Gestionado frente a GPU

Fórmula matemática para determinar el volumen de tokens mensual que justifica un servidor dedicado

Fórmula del Punto de Equilibrio (Break-Even Point)

Volumen de Cruce (Tokens/Mes) = Costo Fijo Servidor ($ USD/Mes) / Costo Promedio Token API

Frente a Modelo de Frontera ($3 / $15 por 1M)

Costo mixto: ~$6.00 USD por 1M tokens

g5.xlarge ($730/mes): Cruce en ~120M tokens/mes

Superados 120M tokens, el servidor propio ahorra dinero

Frente a Modelo Ligero ($0.035 / $0.14 por 1M)

Costo mixto: ~$0.06 USD por 1M tokens

g5.xlarge ($730/mes): Cruce en ~12.000M tokens

Más económico mantener el API Serverless

Regla FinOps: Justificar GPU propia solo si la utilización sostenida supera el 70 % diario

Perfil de Latencia y SLAs: Gestionado vs GPU

Comportamiento del tiempo de respuesta bajo ráfagas de tráfico y fallos de zona

Servicios Gestionados (MaaS)

Acuerdos de Nivel de Servicio
SLA de 99.9 % de disponibilidad respaldado por contrato

Comportamiento en Ráfagas
Absorbe picos sin degradar latencia si está bajo cuota

Riesgo Residual
Jitter en horas pico regionales por concurrencia compartida

Servidores Dedicados (Self-Hosted)

Acuerdos de Nivel de Servicio
SLA autogestionado dependiente del diseño del clúster

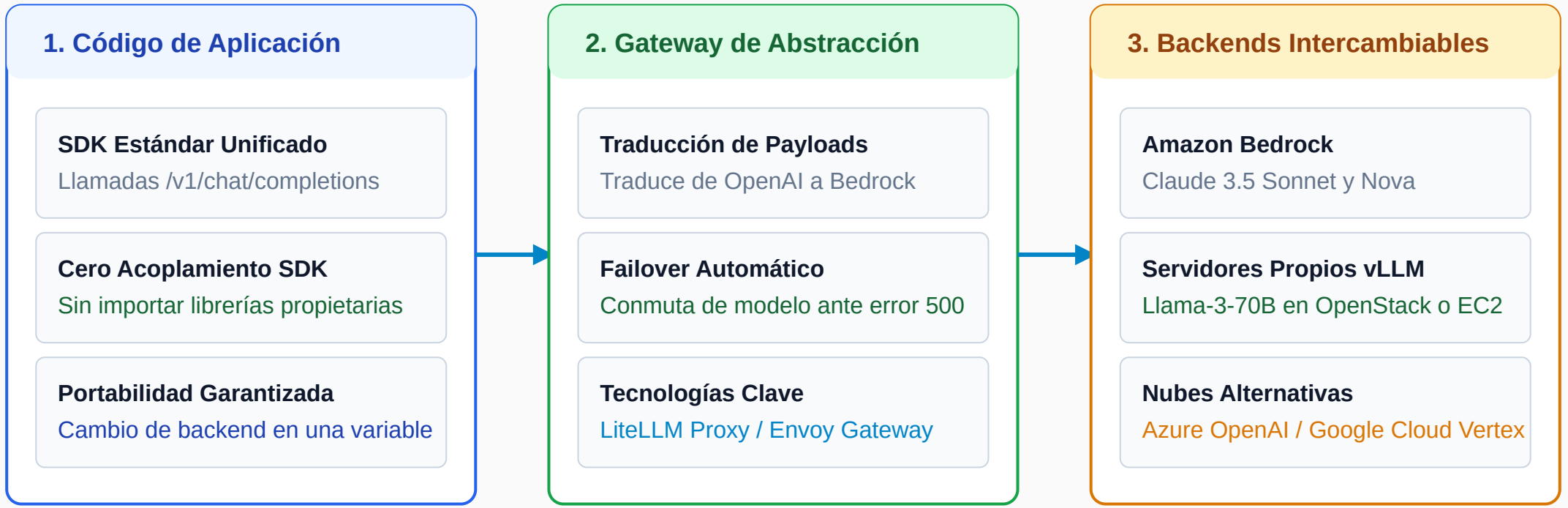
Comportamiento en Ráfagas
P99 se degrada si la cola de peticiones supera la capacidad

Ventaja Principal
Latencia mínima garantizada para tráfico nominal local

Fuente: AWS Service Level Agreements & Real-world AI Benchmark Analysis (2025)

Mitigación del Bloqueo de Proveedor (Vendor Lock-in)

Capas de abstracción y adopción de la especificación OpenAI API para desacoplar el código



Fuente: The Open-Source AI Gateway Framework & LiteLLM Documentation (2025)

ARQUITECTURA HÍBRIDA

Ruteo Dinámico Inteligente de Inferencia

Enrutamiento selectivo por complejidad: modelos ligeros para el 70 % y modelos avanzados para el resto



Fuente: RouteLLM Framework & Berkeley Sky Computing Lab (2024)

MARCO NORMATIVO

Soberanía de Datos y Cumplimiento en Colombia

Exigencias regulatorias de la Ley 1581 de 2012 y Circulares de la Superfinanciera para IA

1. Ley 1581 de 2012

Transferencia Internacional
Prohibida sin nivel adecuado de país

Consentimiento Expreso
Autorización para procesar con IA

Sanciones SIC
Multas hasta 2.000 SMMLV

2. SFC (Circular 007)

Cómputo en la Nube
Gestión estricta de riesgo operativo

Custodia de Llaves
Control exclusivo de cifrado KMS

Trazabilidad Total
Pistas de auditoría inmutables

3. Solución Arquitectónica

Anonimización Previa
Enmascara cédulas y nombres

Enrutamiento Híbrido
Datos privados en servidor local

Blindaje Legal
Sin riesgo de sanciones de la SIC

Fuente: Ley 1581 de 2012 — Congreso de Colombia & Circular Externa 007 de la SFC (2025)

GUÍA DE DECISIÓN

Matriz de Decisión: ¿Gestionado o Auto-Alojado?

Árbol de criterios arquitectónicos para seleccionar la estrategia de despliegue óptima

API Gestionada (Bedrock)

Perfil de Proyecto

Fase inicial, pilotos, RAG
o tráfico esporádico

Ventaja Clave

Cero costo de mantenimiento
y acceso a modelos frontera

Elección Óptima < 50M tok/mes

Arquitectura Híbrida (Router)

Perfil de Proyecto

Aplicaciones a gran escala
con tráfico heterogéneo

Ventaja Clave

Despacha 70 % a modelo bajo
costo y 30 % a frontera

Mayor ROI en producción

Auto-Alojado (vLLM)

Perfil de Proyecto

Datos ultra-sensibles,
soberanía o tráfico masivo

Ventaja Clave

Control total de pesos,
latencia y costo fijo

Obligado por regulación

Fuente: Guía de Decisión de Inferencia en Nube — Universidad de la Costa (2026)

Conclusiones y Conexión con la Semana 16

De la infraestructura de cómputo y compresión hacia las capacidades avanzadas de agentes y razonamiento

1. Silicio y Memoria

Hardware Acelerado

El ancho de banda HBM3 y los enlaces NVLink determinan el techo de latencia

Hopper / Blackwell

2. Cuantización

Compresión Eficiente

Técnicas AWQ y GPTQ junto con GQA reducen la VRAM 4x sin perder precisión

INT4 y FP8 Estándar

3. Motores vLLM

Inferencia de Nube

PagedAttention y continuous batching maximizan la ocupación del hardware dedicado

Throughput Escala

4. Próxima Semana

Semana 16: IA III

Capacidades avanzadas Razonamiento profundo, programación, agentes y repaso de Unidad 3

Hacia la Autonomía

Fuente: Síntesis de Arquitectura en la Nube — Universidad de la Costa (2026)